Detecting Machine-Generated Text: Models, Algorithms, and an Honest Account of What Worked*

Team Cold Start

July 2026

Abstract

We document the full modelling campaign for a binary classification task: deciding whether a scientific abstract was written by a human or a machine. The report covers a from-scratch logistic regression (Task 1), PCA with k-nearest neighbours (Task 2), and an open model search (Task 3) spanning linear SVMs on TF-IDF representations, test-time adaptation, stacked generalization, gradient-boosted trees, and a fine-tuned transformer. For each model we state *why* it was chosen and give the algorithm used to apply it. We report failures alongside successes: a train→test distribution shift dominated the project, two carefully validated ensembles deflated by ≈ 0.075 macro F1 between our validation proxy and the real leaderboard, and quantifying that deflation became our most valuable result.

1 Problem and Data

Given a scientific abstract, predict $y \in \{0, 1\}$ (1 = machine-generated). Provided: 20,000 labelled training abstracts (62.5% machine), 6,999 unlabelled test abstracts, and a fixed 5,000-dimensional TF-IDF feature matrix for both. Evaluation is macro-averaged F1:

$$\text{Macro-F1} = \tfrac{1}{2}\left(\text{F1}_{\text{class 0}} + \text{F1}_{\text{class 1}}\right), \qquad \text{F1}_c = \frac{2\,P_c R_c}{P_c + R_c}, \tag{1}$$

with P_c, R_c the precision and recall of class c. Macro averaging weights the minority (human) class equally with the majority class, which is why several of our models use class weighting.

Two facts we established empirically shaped everything:

1. **No development set exists** (contrary to the brief); we substituted a 10% stratified holdout and, later, k-fold schemes.

2. **The test set is distribution-shifted from train.** A classifier trained to separate train rows from test rows (*adversarial validation*) reaches AUC 0.81 on character n-grams, and test abstracts are ∼25% longer. This is covariate shift in the sense of Shimodaira [23]. Our single-holdout validation score of 0.8229 corresponded to a real leaderboard score of 0.7299 — a deflation of −0.09 that no amount of ordinary cross-validation detects, because CV folds share the training distribution.

*50.007 Machine Learning (May 2026), Kaggle competition `50-007-machine-learning-may-2026`, metric: Macro F1. Team *Cold Start*. Standing public leaderboard score at time of writing: 0.72990 (2nd place; 1st place: 0.78326).

2 Task 1: Logistic Regression from Scratch

2.1 Why this model

Logistic regression is the canonical probabilistic linear classifier: it models $\Pr(y{=}1 \mid \mathbf{x})$ directly, its negative log-likelihood is convex (so gradient descent finds the global optimum), and on a *fixed* high-dimensional TF-IDF representation a linear decision boundary is a strong prior — text classes tend to be close to linearly separable in sparse lexical spaces [11]. It is also the required model for Task 1; the from-scratch constraint means every component below is our own NumPy code, with sklearn used only as a benchmark.

2.2 Model and gradients

With weights $\mathbf{w} \in \mathbb{R}^{5000}$, bias b, and the sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$:

$$\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b), \qquad \mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big], \tag{2}$$

whose gradients have the classic closed form

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{m} X^\top (\hat{\mathbf{p}} - \mathbf{y}), \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \sum_i (\hat{p}_i - y_i). \tag{3}$$

Numerical safeguards: logits clipped to ± 500 before the sigmoid; probabilities clipped to $[10^{-9}, 1 - 10^{-9}]$ inside the logarithm.

2.3 How it is applied

Algorithm 1: *Mini-batch gradient descent for logistic regression (Task 1)*

1. $\mathbf{w} \leftarrow \mathbf{0}$, $b \leftarrow 0$ ▷ zero init is fine: the objective is convex
2. **for** epoch $= 1 \dots 500$:
   (a) shuffle training indices with a seeded RNG
   (b) **for** each mini-batch $(X_B, \mathbf{y}_B)$ of size 512: $\hat{\mathbf{p}} \leftarrow \sigma(X_B \mathbf{w} + b)$; $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{|B|} X_B^\top (\hat{\mathbf{p}} - \mathbf{y}_B)$; $b \leftarrow b - \eta \cdot \frac{1}{|B|} \sum (\hat{\mathbf{p}} - \mathbf{y}_B)$ ▷ $\eta = 10.0$
3. predict $\hat{y} = \mathbb{1}[\sigma(\mathbf{x}^\top \mathbf{w} + b) \geq 0.5]$

The learning rate $\eta = 10$ is unusually large by textbook standards and deliberate: on rows of a (partially normalized) sparse TF-IDF matrix, per-feature gradient magnitudes are tiny, and plain SGD needs an aggressive step to converge in 500 epochs (final training loss ≈0.30).

2.4 Results and two failed refinement rounds

Model	Val macro F1 (90/10, seed 42)	Kaggle public
Ours (from scratch)	**0.7318**	**0.68578**
sklearn `LogisticRegression` (benchmark)	0.7061	—

Round 1 (all FAIL): L2 grid (best +0.0014 at $\lambda{=}5{\times}10^{-4}$), class weighting ($\approx 0$), Adam [14] with column standardization (-0.01 to -0.02: standardization destroys TF-IDF sparsity structure), learning-rate decay, 800 epochs, and honest threshold tuning (+0.0006; threshold chosen on an inner 85/15 split, never on validation). All gains sit inside the 0.022 seed-to-seed

noise band measured over 5 splits. A detail worth recording: with $\eta = 10$, effective end-of-training weight shrinkage compounds as $\exp(-\eta\lambda \cdot \text{epochs/bs})$, so useful λ values are an order of magnitude below textbook defaults — naive grids simply collapse the model.

Round 2 (all FAIL): the provided feature rows are *not* unit-norm (row norms 0.38–1.00; the matrix was evidently normalized before truncation to 5,000 columns), so we tested row re-normalization, $\log(1 + x)$ sublinear scaling [17], and a 3-seed probability-averaged ensemble. Best candidate: +0.0002 mean over 5 seeds.

Conclusion: 0.7318 is the capacity ceiling of a linear boundary on these fixed features; eleven levers across two rounds converge on zero. This also rules out "distilling" a stronger teacher into this model — distillation transfers knowledge, not capacity.

3 Task 2: PCA + k-Nearest Neighbours

3.1 Why this model

PCA [18, 12] is the optimal linear projection in the least-squares sense: it keeps the directions of maximal variance, discarding low-variance directions that, for TF-IDF features, are dominated by rare-term noise. k-NN [5] is the natural partner experiment: it makes *no* parametric assumption, so its accuracy directly measures how much class-discriminative geometry survives the projection — exactly what Task 2 asks us to study.

3.2 How it is applied

Algorithm 2: *PCA + k-NN pipeline (Task 2)*

1. center features: $\tilde{X} = X - \bar{\mathbf{x}}$ ▷ mean from train only
2. compute top-d eigenvectors V_d of $\frac{1}{m}\tilde{X}^\top\tilde{X}$ (equivalently, truncated SVD)
3. project: $Z_{\text{train}} = \tilde{X}V_d, \quad Z_{\text{test}} = (X_{\text{test}} - \bar{\mathbf{x}})V_d$
4. **for** each test point $\mathbf{z}$: find the k=2 nearest training points by Euclidean distance; predict the majority label

3.3 Results: the dimensionality curve

PCA components d	100	500	1000	2000
Kaggle public F1	**0.67793**	0.66373	0.55923	0.41495

Monotone degradation with d is the curse of dimensionality in action: as d grows, pairwise distances concentrate and nearest-neighbour votes approach noise [1]. Fewer components denoise the space. Notably, these models scored *above* their validation estimates on the leaderboard — dense projections never see topic vocabulary, foreshadowing §4.2.

4 Task 3: Open Model Search

4.1 Baseline: TF-IDF + Linear SVM

4.1.1 Why this model

For high-dimensional sparse text, linear SVMs are the classical strongest baseline [11, 7]: the hinge loss maximizes the margin, which controls generalization independently of dimensionality [4]. Our representation concatenates *word 1–2 grams* (topic and phrasing) with *character 3–5 grams*

within word boundaries (`char_wb`): character n-grams capture sub-word style — morphology, punctuation habits, generator tokenization artifacts — rather than topics, and proved the most valuable single design choice of the project. Sublinear TF ($1 + \log \mathrm{tf}$) and $\min \mathrm{df} = 2$ pruning follow standard IR practice [17, 20].

4.1.2 How it is applied

The soft-margin linear SVM solves

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{m} c_{y_i} \max\big(0,\, 1 - y'_i(\mathbf{w}^\top \mathbf{x}_i + b)\big), \qquad y'_i \in \{-1, +1\}, \tag{4}$$

with $C = 0.25$ (strong regularization — chosen by validation, and consistent with a shifty test set) and class weights c_y inversely proportional to class frequency (`class_weight='balanced'`) to protect macro F1's minority-class term. Optimization is LIBLINEAR's dual coordinate descent [7].

Result: validation 0.8229 → Kaggle 0.72990 — our standing best, and the discovery of the central problem.

4.2 The distribution shift, and a validation proxy

The −0.09 deflation reproduces under vanilla 5-fold CV (0.8189), so it is bias, not variance. Diagnosis (adversarial AUC 0.81; test abstracts ∼25% longer) implies vanilla CV cannot rank models for this leaderboard. We built a **cluster-holdout proxy**: k-means [16] with 10 clusters on word-unigram TF-IDF, merged into 5 balanced groups; each fold holds out *entire topic groups*, simulating unseen research areas. The baseline scores 0.7383 under this proxy, within 0.008 of its real 0.7299 — a validated leaderboard estimator *for single text models* (the qualification matters; see §4.7).

4.3 Failure #1: the shift-aware ensemble (submitted)

A soft-vote of a Platt-calibrated [19] char-only logistic regression and a LightGBM over 10 stylometric features (pruned from 18 by adversarial AUC after we caught the raw set being nearly as shift-prone as text itself). Proxy 0.7884; **real leaderboard** 0.7135 — below baseline. Deflation −0.075.

4.4 Test-time adaptation: working backwards from the test set

4.4.1 Why these methods

If the failure mode is a mismatch between the training and deployment distributions, the principled fixes change the *training distribution*, not the model class: importance weighting is the classical estimator-correcting response to covariate shift [23, 24], and self-training/pseudo-labelling [22, 15] lets the model adapt to test-region structure using its own confident predictions. Both use only the unlabelled test *text*, which is legitimately available at prediction time.

4.4.2 How importance weighting is applied

Under covariate shift, the risk on the test distribution is recovered by weighting each training example by the density ratio $w(\mathbf{x}) = p_{\text{test}}(\mathbf{x})/p_{\text{train}}(\mathbf{x})$. We estimate it discriminatively:

Algorithm 3: *Adversarial importance weighting*

1. build char 3–5 gram TF-IDF over train ∪ test text

2. label domain $d_i = 0$ (train) or 1 (test); fit logistic regression for $\Pr(d{=}1 \mid \mathbf{x})$
3. obtain *out-of-fold* probabilities $\hat{q}_i$ for every train row (3-fold CV, so no row is scored by a model that saw it)
4. $w_i \leftarrow \mathrm{clip}\big(\hat{q}_i/(1-\hat{q}_i),\, 0.25,\, 4\big)$; normalize to mean 1 ▷ clipping bounds variance
5. refit the baseline SVM of §4.1 with per-sample weights w_i

4.4.3 How pseudo-labelling is applied

Algorithm 4: *Self-training with confidence margin*

1. fit baseline SVM on labelled train; compute decision scores s_j on unlabelled test
2. select $\mathcal{C} = \{j : |s_j| \geq 1.0\}$ ▷ outside the margin = confident
3. assign $\tilde{y}_j = \mathbb{1}[s_j > 0]$ for $j \in \mathcal{C}$; refit on train $\cup \{(\mathbf{x}_j, \tilde{y}_j)\}_{j \in \mathcal{C}}$

Because our proxy's held-out folds *have* true labels, we could measure pseudo-label quality directly: **98.1% of pseudo-labels were correct** at 8% coverage — self-training feeds the model truth, not noise.

4.4.4 Results (cluster-holdout proxy, 5 folds)

Technique	Proxy F1	Δ vs baseline	Verdict
Baseline (reproduced)	0.7383	—	sanity check passed
Transductive vocabulary	0.7444	+0.006	free, safe
Pseudo-labelling (m=1.0)	0.7444	+0.006	PASS (small)
Importance weighting	**0.7523**	**+0.014**	**PASS: positive on all 5 folds**
IW + pseudo-labelling	0.7524	+0.0001 over IW	no compounding: same signal

4.5 Failure #2: stacked generalization (submitted)

4.5.1 Why this model

Stacking [26] is the principled version of "multiple layers of prediction": train diverse base models, then a meta-learner on their *out-of-fold* predictions — OOF is the leakage guard, since a meta-learner fit on in-sample base predictions merely learns which base model memorizes best. Base-model *diversity* is what the meta-learner monetizes, so we selected members by the correlation matrix of their OOF scores: our proven SVM and a ridge classifier [10] on the same representation correlate at $r = 0.985$ — duplicates, not diversity — so only ridge was kept, alongside Multinomial Naive Bayes on words ($r \approx 0.63$–0.75 to everything), LightGBM on truncated-SVD [9] + stylometrics, and a PCA-logistic model on the provided features.

4.5.2 How it is applied

Algorithm 5: *Leakage-safe stacking with nested shift-aware evaluation*

1. **for** each base model b (level 0): generate OOF scores via stratified 5-fold — fit on 4 folds (vectorizers/SVD/PCA fit on those folds *only*), score the 5th
2. fit meta-learner (logistic regression / ridge) on the OOF score matrix ▷ level 1
3. **evaluate** on cluster-holdout: within each topic fold, regenerate *inner* OOF on fold-train only, fit meta on that, score the held-out topics
4. **final refit**: bases on all 20,000 rows for test scores; meta on the full-train OOF matrix

4.5.3 Result

Proxy 0.8018 (best single base: 0.7717), train/val gap only +0.055, two meta-learners agreeing within 0.002 — every screen we knew to apply, passed. **Real leaderboard: 0.72407**, below the 0.72990 incumbent. Deflation −0.078.

4.6 Gradient-boosted trees (built, deliberately not submitted)

4.6.1 Why this model

Gradient boosting [8] fits an additive ensemble of trees, each tree approximating the Newton step on the current loss [3]; LightGBM's histogram algorithm [13] makes this fast on dense features. The hypothesis: dense representations (SVD of TF-IDF + stylometrics) might inherit the shift-robustness the PCA+KNN family showed. After ∼30 configurations of XGBoost/LightGBM/random forest [2] over five representations, the winner (LightGBM, 1200 trees, learning rate 0.03, 63 leaves, on SVD-300 + 10 stylometric features) reached proxy 0.8079 — but with train F1 = 1.000 (gap +0.192) and membership in exactly the family that produced both −0.075 deflations. Under the calibration rule below it projects to ∼0.73 real; we withheld the submission.

4.7 The calibration rule: what two failures purchased

Model family	Proxy → real deflation	Evidence
Single sparse-text model	−0.008	SVM: 0.7383 → 0.7299
Stylometric/tree legs, or ensembles thereof	**−0.075 to −0.078**	ensemble: 0.7884 → 0.7135; stack: 0.8

Mechanism: the cluster proxy simulates *topic* shift (held-out vocabulary) but cannot simulate *stylometric* shift — the real test abstracts are ∼25% longer, and length-correlated signal is precisely what stylometric and tree legs exploit. Two independent submissions deflating by the same amount convert suspicion into a usable correction factor.

4.8 Transformer fine-tuning (in progress)

4.8.1 Why this model

Every model above learns from these 20,000 abstracts alone, so topic shift starves it. A pretrained transformer [25, 6] brings language representations learned from billions of tokens; fine-tuning adapts them to the task, and that pretraining is exactly what should transfer across the topic shift. We chose DistilBERT [21] — 40% smaller than BERT with ∼97% of its quality — to fit the compute budget (Apple MPS, no CUDA).

4.8.2 How it is applied

Algorithm 6: *Fine-tuning DistilBERT for sequence classification*

1. tokenize abstracts with the pretrained WordPiece tokenizer; truncate/pad to $L = 256$ tokens
2. attach a 2-way linear head on the `[CLS]` representation
3. train *all* parameters end-to-end: AdamW, lr 2×10^{-5}, linear warmup over the first 6% of steps then linear decay, weight decay 0.01, batch 16, 2 epochs
4. evaluate macro F1 on the holdout after each epoch; keep the best-epoch checkpoint
5. final: retrain on all 20,000 rows; predict test with arg max over the head's logits

The small learning rate and warmup are not optional niceties: fine-tuning at SGD-scale rates catastrophically overwrites pretrained weights, and warmup protects them while the fresh head's gradients are still large [6].

4.8.3 Epoch-level results

	Epoch 1	Epoch 2
Stratified 90/10 holdout macro F1	**0.8498** (selected)	0.8393

Train F1 0.9067, gap +0.057 — an order of magnitude below the classical models' ∼0.25 memorization gap. On the shift proxy (identical fold definitions as every experiment above):

Cluster fold	DistilBERT	SVM baseline (same fold)	Δ
0 (largest, hardest)	0.7580	0.7331	+0.025
1	0.8740	0.7911	+0.083
mean (2 folds)	**0.8160**	0.7621	+0.054

Known weakness: 66% of test abstracts exceed 256 tokens (test median 345 vs train 245 — the length shift again); a max-length-448 rerun is training at the time of writing. As a non-ensemble, non-stylometric family with a small gap that beats the baseline on every fold tested, it is our best remaining candidate against the 0.78326 target.

5 Complete Submission Record

#	Submission	Public F1	Verdict
1–4	PCA+KNN ($d = 2000/1000/500/100$)	0.415/0.559/0.664/**0.678**	Task 2 curve — PASS
5	**LinearSVC word+char TF-IDF**	**0.72990**	**standing best**
6	Shift-aware soft-vote ensemble	0.71351	FAIL (proxy overrated)
7	Task 1 from-scratch logistic regression	0.68578	PASS vs sklearn benchmark
8	From-scratch TF-IDF + logistic regression	0.67283	FAIL (mindf= 1 vocab overfit)
9	From-scratch gradient-boosted trees	0.71464	below baseline
10	Stack (4 diverse bases, logistic meta)	0.72407	FAIL (proxy overrated, again)

Kaggle scores submissions on a hidden public subset and keeps the team's best, so probes 6–10 cost nothing but information — and #6 and #10 bought the calibration rule of §4.7.

6 Lessons, Ranked by What They Cost

1. **Validation is a model of deployment, and it can be wrong in structured ways.** Our proxy was accurate for single text models and systematically ∼0.075 optimistic for stylometric/ensemble families; we learned the difference only by submitting, but we learned it as a *number*.

2. **Distribution shift dominates model choice.** Representation (`char_wb` n-grams, mindf pruning) and training distribution (importance weighting) moved the leaderboard more than any classifier swap.

3. **Capacity ceilings are measurable.** Eleven levers, two rounds, five seeds each: the Task-1 model does not move. Knowing a model is *done* is as valuable as improving it.

4. **Pseudo-labels can be audited before being trusted** (98% precision here) — though their gain was already captured by importance weighting; the two share one signal.

5. **Prediction-correlation matrices are the right diversity screen for ensembles** (r=0.985 means duplicate, not diverse) — even when the ensemble family itself proves shift-fragile.

References

[1] K. Beyer, J. Goldstein, R. Ramakrishnan, U. Shaft. When is "nearest neighbor" meaningful? *ICDT*, 1999.

[2] L. Breiman. Random forests. *Machine Learning*, 45(1), 2001.

[3] T. Chen, C. Guestrin. XGBoost: A scalable tree boosting system. *KDD*, 2016.

[4] C. Cortes, V. Vapnik. Support-vector networks. *Machine Learning*, 20(3), 1995.

[5] T. Cover, P. Hart. Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory*, 13(1), 1967.

[6] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL*, 2019.

[7] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, C.-J. Lin. LIBLINEAR: A library for large linear classification. *JMLR*, 9, 2008.

[8] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 2001.

[9] N. Halko, P.-G. Martinsson, J. A. Tropp. Finding structure with randomness. *SIAM Review*, 53(2), 2011.

[10] A. E. Hoerl, R. W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 1970.

[11] T. Joachims. Text categorization with support vector machines. *ECML*, 1998.

[12] I. T. Jolliffe. *Principal Component Analysis*, 2nd ed. Springer, 2002.

[13] G. Ke et al. LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS*, 2017.

[14] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. *ICLR*, 2015.

[15] D.-H. Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. *ICML Workshop*, 2013.

[16] J. MacQueen. Some methods for classification and analysis of multivariate observations. *Berkeley Symposium*, 1967.

[17] C. D. Manning, P. Raghavan, H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[18] K. Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11), 1901.

[19] J. Platt. Probabilistic outputs for support vector machines. *Advances in Large Margin Classifiers*, 1999.

[20] G. Salton, C. Buckley. Term-weighting approaches in automatic text retrieval. *Inf. Processing & Management*, 24(5), 1988.

[21] V. Sanh, L. Debut, J. Chaumond, T. Wolf. DistilBERT, a distilled version of BERT. *NeurIPS Workshop*, 2019.

[22] H. Scudder. Probability of error of some adaptive pattern-recognition machines. *IEEE Trans. Inf. Theory*, 11(3), 1965.

[23] H. Shimodaira. Improving predictive inference under covariate shift by weighting the log-likelihood function. *J. Statistical Planning and Inference*, 90(2), 2000.

[24] M. Sugiyama, M. Krauledat, K.-R. Müller. Covariate shift adaptation by importance weighted cross validation. *JMLR*, 8, 2007.

[25] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

[26] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5(2), 1992.